

A3-PS036-TC529

PQC DNSSEC MTL Benchmark Using a Reference Implementation

Team CLIMB

Pratyay Mukherjee: pratyay.bsmsfs4232806@nfsu.ac.in

Mohit Kumar Yadav: mohityadav151217@gmail.com

Dr. Maibam Devi (Mentor): maibam.devi@nfsu.ac.in

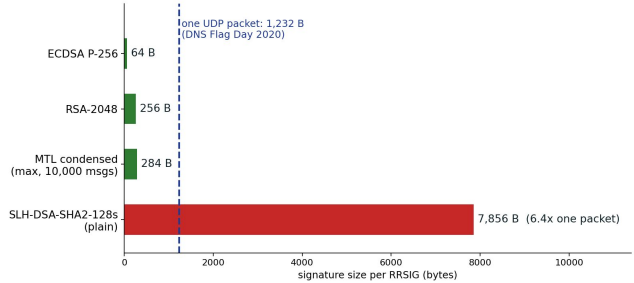
AIORI Hackathon 3

October 14, 2026

Abstract / Problem Description

PQC signatures do not fit in one DNS packet. MTL mode may fix that, and PS-036 asks us to measure the trade-off.

- DNSSEC authenticates DNS answers with digital signatures (RRSIG). Today's signatures are small: ECDSA P-256 = 64 B, RSA-2048 = 256 B. [1]
- A large quantum computer would break RSA and ECDSA, so DNSSEC must move to post-quantum (PQC) signatures. SLH-DSA (NIST FIPS 205, hash-based) is the conservative choice, but one SLH-DSA-SHA2-128s signature is 7,856 B. [2]
- DNS answers should fit one UDP packet: 1,232 B per DNS Flag Day 2020 (RFC 9715 recommends at most 1,400 B). One signature alone is 6.4× that. [4]



- An oversized answer is truncated (TC bit) or fragmented and the resolver must retry over TCP: extra round trips, extra bytes, more server load, on every lookup, because every answer carries its own signature. [3]
- Merkle Tree Ladder (MTL) mode signs a small "ladder" of Merkle roots once; each record then carries only a short Merkle proof, so the signature cost is amortised over a batch. [5][7]
- Open question for PS-036: how much does MTL mode save in signing time, verification time, transmitted bytes and cache usage, and what does it cost (ladder fetch, hashing, cache dependency)?
- Published analyses exist [7][8]. This problem asks for an independent, reproducible benchmark on the Verisign reference implementation, with and without MTL, on three dataset sizes.

Background

DNSSEC [1]

Every RRset is returned with an RRSIG that a resolver verifies with the zone's DNSKEY, up the chain to a trust anchor. The signature travels in every answer, so its size is paid on every lookup.

The UDP size limit [3][4]

Resolvers advertise an EDNS(0) buffer. DNS Flag Day 2020 recommends 1,232 B (1,280-byte IPv6 minimum MTU minus 40 and 8); RFC 9715 recommends at most 1,400 B. A bigger answer sets TC=1 and is re-requested over TCP.

SLH-DSA-SHA2-128s [2]

NIST FIPS 205, hash-based, security level 1. Public key 32 B, secret key 64 B, signature 7,856 B.

MTL mode [5][6][7]

The signer signs a ladder (a few Merkle roots) instead of every message. Each message carries a condensed signature (a Merkle authentication path). The signed ladder is a full signature that a resolver fetches once and caches. A resolver first queries without the mtl-mode-full option and re-queries with it, preferably over TCP, only if no cached ladder covers the proof. [6]

Item	Size
ECDSA P-256 signature	64 B
RSA-2048 signature	256 B
SLH-DSA-SHA2-128s signature	7,856 B (6.4 x one packet)
MTL condensed signature, 10,000 msgs	140-284 B (mean 273.5 B)
MTL signed ladder, 10,000 msgs	8,056 B, once per ladder

Sizes we measure

Algorithm measured: SLH-DSA-SHA2-128s in MTL mode (SLH-DSA-MTL-SHA2-128s), hash size 16 B. Condensed signature = 76 B + 16 B per Merkle-path step.

Literature & RFC Review 1

Eight sources define what we build, what we measure and what we compare against.

Source	What it defines	How CLIMB uses it
NIST FIPS 205 [2]	SLH-DSA; SHA2-128s sizes (32 B key, 7,856 B signature)	The algorithm under test; the plain-SLH-DSA baseline
draft-harvey-cfrg-mtl-mode-09 [5] (24 Mar 2026)	MTL mode: ladder, condensed and full signatures; SLH-DSA and ML-DSA instances	Ground truth for our analytical ladder model, verified byte-for-byte against libMTL
draft-fregly-dnsop-slh-dsa-mtl-dnssec-06 [6] (30 Mar 2026)	SLH-DSA-MTL in DNSKEY/RRSIG; MTL-Type byte (0 condensed, 1 full); mtl-mode-full EDNS(0) option; batch signing	RRSIG wire format and the cold/warm query flow we test
Fregly, Harvey, Kaliski, Sheth, CT-RSA 2023 [7]	Design and size analysis of MTL mode	The hypothesis our benchmark tests

Literature & RFC Review 2

Eight sources define what we build, what we measure and what we compare against.

Source	What it defines	How CLIMB uses it
Tran and Chung, IETF 122 [8]	Randomised evaluation of SLH-DSA-MTL signature sizes	Closest prior evaluation, focused on sizes; CLIMB adds timing, cache and live-resolver measurements in a reproducible pipeline
DNS Flag Day 2020; RFC 9715; RFC 6891, 7766, 9210 [3][4]	1,232 / 1,400 B UDP limits; EDNS(0); TCP fallback	UDP-fit thresholds for the size analysis
Verisign Labs libMTL and MTL-mode forks of ldns, NSD, Unbound [9]	Reference library and the DNS software the SLH-DSA-MTL draft lists as its implementations	What we benchmark: pinned fork, called from Rust through FFI and run in a live Docker pipeline
ML-DSA-MTL drafts incl. SigTag (kaizer; hdong-...-sigtag-ext-00, 28 Sep 2026) [11]	ML-DSA instance of MTL for DNSSEC; SigTag EDNS(0) option for signalling known ladders	Out of scope (we test SLH-DSA); noted as the extension path

Internet-Drafts are work in progress; revisions checked on datatracker on 14 Oct 2026.

Proposed Solution

CLIMB: a reproducible benchmark on the real MTL reference code, cross-checked on live DNS software.

- - What: CLIMB (Cryptographic Ladder Impact Measurement Benchmark) measures SLH-DSA-SHA2-128s in MTL mode on three seeded datasets: Small = 100, Medium = 10,000, Large = 1,000,000 messages.
- - Three configurations, defined once and used on every slide:

1. Plain SLH-DSA: one 7,856 B signature per message (no MTL).
2. MTL without amortisation: every message carries its condensed proof plus a full signed ladder.
3. MTL amortised: a condensed proof per message plus one signed ladder per batch.

- - Metrics (the four required by PS-036): signing time, verification time (ladder cached or not), transmitted bytes, cache and memory footprint.
- - Method, branch 1: a Rust harness binds directly to the C reference library (libMTL v1.2.1, pinned fork) through FFI; Criterion statistics; deterministic seeds; fingerprinted environment.
- - Method, branch 2: a Docker pipeline with Verisign's MTL-mode ldns signer, NSD authoritative server and Unbound resolver serves a real signed zone and captures real wire responses.
- - Evidence discipline: every number is labelled MEASURED, ANALYTICAL, EXTRAPOLATED or DERIVED and never blended; report numbers are registered claims checked against the data (report --check); 420+ automated tests; one command (cargo xtask reproduce) regenerates everything; a terminal dashboard (climb-tui) browses the results.
- - Scope (per PS-036): no new cryptography or security proof, no resolver written from scratch, no public-Internet deployment.

Solution Optimization 1

Pay the post-quantum signature cost once per batch, and give every record a short Merkle proof.

- One SLH-DSA signature per batch (the ladder) instead of one per message.
- A condensed signature (76 B + 16 B per tree level) replaces a 7,856 B signature in every answer.
- The signed ladder (about 8 KB) is fetched once and cached by the resolver.

Dataset	A: plain SLH-DSA	B: MTL, no amortisation	C: MTL amortised	C vs A	C vs B
Small (100)	785.6 KB	815.6 KB	24.4 KB	−96.9% (32.2×)	−97.0% (33.4×)
Medium (10,000)	78.56 MB	83.30 MB	2.74 MB	−96.5% (28.6×)	−96.7% (30.4×)
Large (1,000,000)	7,856 MB	8,487 MB	367.1 MB	−95.3% (21.4×)	−95.7% (23.1×)

B and C: wire-size harness, matching the analytical ladder model byte-for-byte. A = N × 7,856 B (FIPS 205). C includes one signed ladder. 1 MB = 10^6 B.

Solution Optimization 2

Metric	Before	After	Improvements	Evidence class
Signing 10,000 messages	10,000 SLH-DSA signatures (about 0.7 s each)	1 ladder signature + Merkle-tree hashing	≈ 9,869× faster	EXTRAPOLATED baseline
Verification per message	full SLH-DSA verification	Merkle-path check against a cached ladder	≈ 83× faster	MEASURED
Live DNS answer (www.climb.example. A)	full-size signatures: 24,438 B (exceeds 1,232 B)	condensed signatures: 658 B (fits one UDP packet)	−97.3% (≈ 37×)	MEASURED on the wire
Signature storage, zone DB / resolver cache (10,000 RRSIGs)	78.56 MB	2.74 MB	−96.5% (28.6×)	ANALYTICAL

Signing baseline: N independent SLH-DSA signatures, extrapolated from sampled signatures (infeasible to run in full on the reference machine). First answer for a new ladder (ladder not cached) must verify the ladder signature and costs about one full verification; the gain starts once the ladder is cached.

Solution Architecture & Design

AIORI 3Architecture Design (UML Diagram)

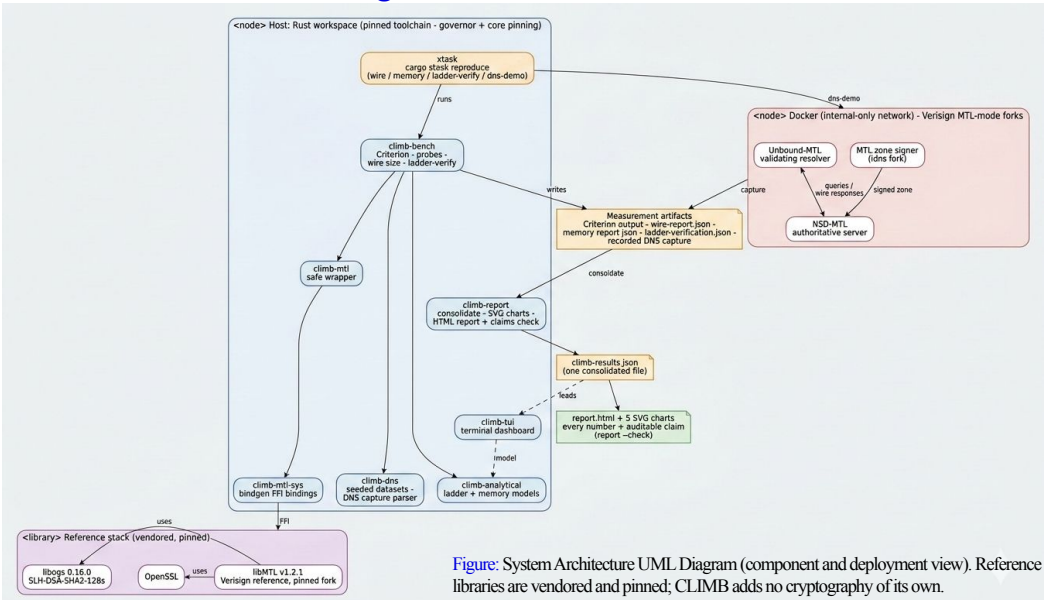


Figure: System Architecture UML Diagram (component and deployment view). Reference libraries are vendored and pinned; CLIMB adds no cryptography of its own.

Data Flow Diagram

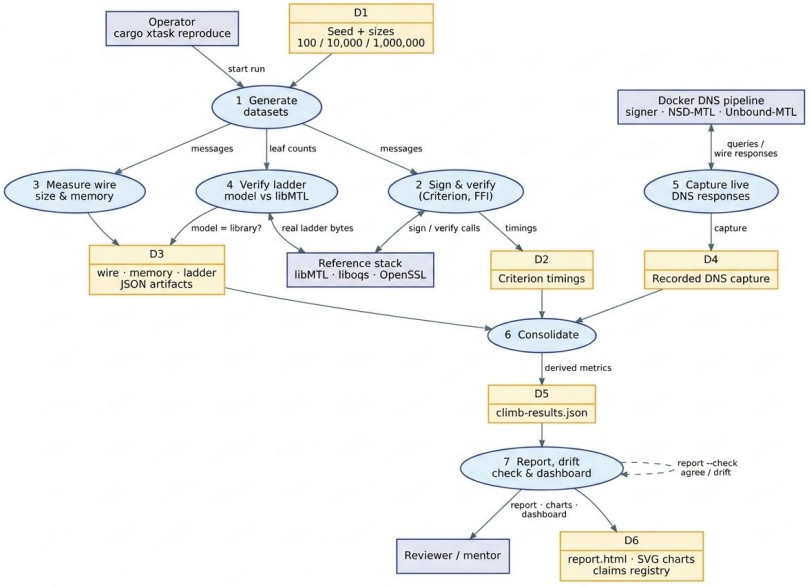


Figure: Data Flow Diagram (DFD), level 1. Rectangles: external entities. Ellipses: processes. Two-cell boxes: data stores. Dashed loop: the claims check (report --check) that fails if the report drifts from the data.

Deployment Timeline

Phases 1–5 and the dashboard are complete; the live demonstration, extra analyses and documentation follow.

- **Phase 1: FFI foundation (Done)**
  - Bindgen FFI to libMTL, safe Rust wrapper, round trips against real test vectors.
- **Phase 2: Workloads and timing (Done)**
  - Seeded Small / Medium / Large datasets; sign and verify with and without MTL.
- **Phase 3: Bytes, memory, environment (Done)**
  - Wire size, memory (measured and analytical), environment fingerprint.
- **Phase 4: Live DNS pipeline (Done)**
  - Docker: ldns signer, NSD-MTL, Unbound-MTL; real signed zone; wire capture.
- **Phase 5: Report and dashboard (Done)**
  - Consolidated results, charts, HTML report with claim checking, terminal dashboard, ladder model verified against libMTL for 17 sizes.
- **Phases 6–7: Demo, analyses, documentation (Planned)**
  - Polished live demonstration; cold vs warm ladder reuse; UDP fit and TCP fallback; plain SLH-DSA and classical baselines; documentation; mentor review of the MTL implementation.

Deployment Methodology

- One command. ‘cargo xtask reproduce’ rebuilds every artifact from pinned sources.
- Gated phases. Every phase passes build, format, lint (warnings denied) and tests (420+) before the next starts.
- Labelled evidence. **MEASURED**, **ANALYTICAL**, **EXTRAPOLATED**, **DERIVED**: never blended.
- Checked claims. Report numbers are registered and verified against the data automatically.
- Self-describing runs. Each run records its environment fingerprint, so any number can be traced and re-run.

Bibliography I

[1] R. Arends, R. Austein, M. Larson, D. Massey, S. Rose. RFC 4033, 4034, 4035 (DNSSEC), March 2005. P. Hoffman, W. Wijngaards. RFC 6605 (ECDSA for DNSSEC), 2012.
[2] NIST. FIPS 205: Stateless Hash-Based Digital Signature Standard. 13 August 2024. doi:10.6028/NIST.FIPS.205.
[3] J. Damas, M. Graff, P. Vixie. RFC 6891: Extension Mechanisms for DNS (EDNS(0)), 2013. RFC 7766 and RFC 9210: DNS Transport over TCP.
[4] DNS Flag Day 2020, dnsflagday.net/2020 (1,232-byte EDNS buffer). RFC 9715: IP Fragmentation Avoidance in DNS over UDP, January 2025.
[5] J. Harvey, B. Kaliski, A. Fregly, S. Sheth, D. McVicker. Merkle Tree Ladder (MTL) Mode Signatures. Internet-Draft draft-harvey-cfrg-mtl-mode-09, 24 March 2026 (work in progress).
[6] A. Fregly, J. Harvey, B. Kaliski, D. Wessels. SLH-DSA-MTL for DNSSEC. Internet-Draft draft-fregly-dnsop-slh-dsa-mtl-dnssec-06, 30 March 2026 (work in progress).
[7] A. Fregly, J. Harvey, B. S. Kaliski Jr., S. Sheth. Merkle Tree Ladder Mode: Reducing the Size Impact of NIST PQC Signature Algorithms in Practice. CT-RSA 2023, LNCS 13871, pp. 415-441, Springer, 2023. ePrint 2022/1730.
[8] M. Tran, T. Chung. Randomized Evaluation of SLH-DSA-MTL's Impact on Reducing PQ-DNSSEC Signature Sizes. IETF 122 pq-dnssec side meeting, 18 March 2025.
[9] Verisign Labs. MTL reference implementation (github.com/verisign/MTL) and MTL-mode forks of ldns, NSD and Unbound (github.com/verisign/mtl-mode-ldns, mtl-mode-nsd, mtl-mode-unbound).
[10] Open Quantum Safe. liboqs 0.16.0. github.com/open-quantum-safe/liboqs.
[11] A. Kaizer, J. Harvey, B. Kaliski, S. Sheth. ML-DSA-MTL for DNSSEC (draft-kaizer-dnsop-ml-dsa-mtl-dnssec). H. Dong, A. Kaizer, J. Harvey, B. Kaliski, S. Sheth. ML-DSA-MTL for DNSSEC with SigTag Extension, draft-hdong-dnsop-ml-dsa-mtl-dnssec-sigtag-ext-00, 28 September 2026 (work in progress; future work).

Thank You for Your Attention!

Team CLIMB — https://github.com/aiori-hackathon/A3-PS036-TC529